

符玄桌面宠物 — 技术文档

符玄桌宠项目

V2.1 • 2026-07-09

摘要

本文档为符玄桌面宠物项目的完整技术报告 (V2.1)。项目基于 Tuanjie 2022.3.62t7 (团结引擎) 与 Live2D Cubism SDK 5-r.4, 构建了一个运行于 Windows 桌面的 AI 驱动虚拟角色。系统深度集成 DeepSeek Chat API (含 Function Calling 37+ 工具)、GLM-4V 多模态视觉模型、以及多层级感知与记忆系统, 实现了从自然语言交互到具身动作表达的完整闭环。V2.1 引入了全新的悬浮球 (BallPanel) 与右侧 Widgets 面板 (RightPanel) 交互体系, 取代了旧版右键菜单, 涵盖物理模拟、Live2D 参数化动画、DWM 透明窗口、AI 对话、天气感知、便签提醒等核心功能。

目录

1	项目概述	4
1.1	项目背景	4
1.2	技术栈	4
1.3	核心指标	4
2	系统架构	5
2.1	分层架构	5
2.2	执行顺序	5
2.3	依赖注入模式	6
3	渲染管道	6
3.1	Live2D Cubism 渲染	6
3.1.1	模型加载双保险	6
3.1.2	参数映射系统	6
3.1.3	Perlin 噪声驱动空闲微动	6
3.1.4	天气表情联动	7
3.2	DWM 透明窗口	7
4	AI 对话系统	8
4.1	DeepSeek Chat API 集成	8
4.1.1	系统提示词注入	8
4.1.2	Function Calling 架构	8
4.1.3	多轮工具调用循环	8

目录	2
4.1.4 异步工具执行	9
4.2 工具索引	9
4.3 句子队列与优先级气泡	10
4.4 自动闲聊系统	10
5 具身动作系统	10
5.1 架构演进	10
5.2 空闲动作系统	11
5.2.1 JSON 配置驱动	11
5.2.2 动作列表	12
5.2.3 插值曲线	12
5.3 LLM 动作翻译 (MotionTranslator)	12
5.3.1 核心流程	12
5.3.2 SPECIAL PATTERNS	13
5.4 动作锁定机制	13
5.5 MotionAgent — 自主动作决策	13
6 闭环具身智能验证	13
6.1 架构设计	13
6.2 MotionVerifier — 动作验证器	14
6.3 VisionMotionVerifier — 视觉验证器	14
6.4 DualModelValidator — 双模型评分	15
6.5 闭环学习 (MotionMemoryManager)	15
7 交互系统	15
7.1 拖拽系统 (DragHandler)	15
7.2 悬浮球辐射菜单 (BallPanel)	16
7.3 右侧 Widgets 面板 (RightPanel)	16
7.4 分区点击反馈	16
7.5 拖拽挣扎动画	16
8 感知与记忆系统	17
8.1 法眼活动追踪 (ActivityTracker)	17
8.2 长期记忆系统 (PetMemory)	17
8.2.1 三层分层结构	17
8.2.2 记忆类别与重要性	17
8.2.3 反思反射	17
8.3 时间与天气系统 (TimeWeatherController)	18
8.3.1 双源天气获取	18
8.3.2 AI 天气语录	18

目录	3
9 桌面物理引擎	18
9.1 核心物理模型	18
9.2 地面任务状态机	18
9.3 多显示器支持	18
9.4 睡眠唤醒检测	19
10 窗口与系统集成	19
10.1 Win32 API 封装	19
10.1.1 DWM 透明窗口	19
10.1.2 系统托盘	19
10.1.3 开机自启	19
10.2 崩溃日志系统	19
10.3 进程互斥	19
11 性能优化	19
11.1 三级性能档位	19
11.2 帧率采样与评估	20
11.3 CPU/GPU 监控	20
11.4 内存管理	20
12 便签与提醒系统	20
12.1 数据模型	20
12.2 推送链路	21
12.3 去重机制	21
13 构建与部署	21
13.1 构建脚本	21
13.2 构建后验证	21
14 版本路线图	22
15 附录	23
15.1 环境变量清单	23
15.2 相关文件路径	23
15.3 关键设计决策	24

1 项目概述

1.1 项目背景

原桌面宠物程序基于 Windows GDI+ 分层窗口技术，使用静态 PNG 图片硬切换模拟动画。本项目将其迁移至 Unity 引擎，利用 Live2D Cubism SDK 实现参数化变形动画，获得更流畅的视觉效果和更丰富的交互能力，并引入 AI 对话、环境感知、具身动作生成等高级特性。版本 V2.1 对交互界面进行了彻底重构，以悬浮球辐射菜单和右侧 Widgets 面板替代了传统右键菜单。

1.2 技术栈

- 引擎：Tuanjie 2022.3.62t7（团结引擎）
- 角色渲染：Live2D Cubism SDK 5-r.4（参数化变形 + 实时物理模拟）
- 3D 渲染：Unity Animator + FBX（预留）
- AI 对话：DeepSeek Chat API + Function Calling（37+ 法术工具）
- 视觉模型：GLM-4V（智谱）— 截图分析 + 动作视觉验证
- 天气数据：QWeather API（优先）/ wttr.in（回退，双源）
- 文件搜索：Everything CLI (es.exe) / 递归回退
- 推送通知：Server 酱³
- 窗口管理：Win32 API（DWM / Shell_NotifyIcon / UIAutomation）
- 构建脚本：PowerShell + Unity Batchmode
- 平台：Windows 10/11 64 位

1.3 核心指标

- 渲染帧率：20–60 fps（自适应三档降频）
- AI 响应延迟：1–5 s（取决于网络与工具调用轮次）
- 内存管理：800 MB 触发 GC，1200 MB 强制 GC
- 系统内存监控：85% 预警 GC，93% 紧急降质
- 工具调用：37+ 注册工具，最多 5 轮回环
- 空闲动作：12 种 JSON 配置驱动 + 2 种硬编码特效

- 动作验证：19 个测试用例，GLM-4V 视觉评分 + 闭环学习
- 长期记忆：核心事实 + Top-5 重要 + 近期琐事，JSON 持久化

2 系统架构

2.1 分层架构

系统采用分层架构设计，从底层到上层依次为：

1	+-----+
2	UI 表示层
3	BallPanel(悬浮) RightPanel(右侧) ChatBubble
4	BottomInputBar DebugWindow
5	+-----+
6	AI 核心层
7	ChatManager ToolCallInvoker IdleChatGenerator
8	MotionTranslator MotionAgent MotionVerifier
9	+-----+
10	感知层
11	ActivityTracker BrowserTabReader PetMemory
12	TimeWeatherController PerformanceMonitor
13	+-----+
14	物理层
15	DesktopPet (重力/碰撞/地面检测) DragHandler
16	+-----+
17	渲染层
18	HybridRenderer -> Live2DRenderer / Model3DRenderer
19	+-----+
20	系统层
21	WindowOverlay (DWM) SystemTrayManager ServerPoll
22	+-----+

2.2 执行顺序

Unity 脚本执行顺序通过 `DefaultExecutionOrder` 属性严格控制：

1. `DesktopPet.Update()` (order=0) — 物理更新、状态转换、行走相位
2. `CubismPhysicsController.LateUpdate()` (order=800) — 衣服/头发/裙子物理模拟

3. Live2DRenderer.LateUpdate() (order=801) — 覆盖被物理重置的参数、空闲动画、交互反馈

顺序 801 确保所有 Live2D 参数在 Cubism Physics 运算后写入，避免物理覆盖关键参数（如手臂摆幅）。

2.3 依赖注入模式

组件间通过 Unity Inspector 序列化引用或单例模式连接。全局单例包括:ChatConfig（环境变量静态访问）、PetMemory.Instance（忆境）、PetConfig.Instance（天机簿）。

3 渲染管道

3.1 Live2D Cubism 渲染

3.1.1 模型加载双保险

Live2DRenderer 使用两级加载策略:第一级加载 AssetDatabase.LoadAssetAtPath（Editor 模式），失败时第二级通过 Resources.Load 降级（构建模式）。

3.1.2 参数映射系统

Live2DParameterMapper 建立语义参数名到 Cubism 参数 ID 的双向映射，涵盖约 80+ 个参数：

```
1 mapper.RegisterParameter("ParamAngleX", "head_angle_x");
2 mapper.RegisterParameter("ParamBodyAngleX", "body_angle_x");
3 mapper.SetParameterValue("head_angle_x", 15f); // 头左倾 15 度
```

Listing 1: 参数映射示例

参数按身体部位分组：头部（3 轴旋转）、身体（3 轴倾角）、眼睛（开合/微笑/眼球）、眉毛（上抬/集中）、嘴巴（形态/张合）、手臂（左右多段）、呼吸。

3.1.3 Perlin 噪声驱动空闲微动

空闲状态由 Perlin 噪声驱动 7 组微动参数，使用不同的种子值确保互不关联：

$$\text{breath} = (\text{PerlinNoise}(t, 0) - 0.5) \times \text{Amplitude}$$

表 1: Perlin 噪声微动参数

参数	通道 (seed)	振幅
ParamBreath	(t, 0)	BREATH_AMPLITUDE
ParamBodyAngleX	(t, 1)	BODY_SWAY_X
ParamBodyAngleY	(t, 2)	BODY_SWAY_Y
ParamBodyAngleZ	(t, 3)	BODY_SWAY_Z
ParamAngleX	(t, 4)	HEAD_X
ParamAngleY	(t, 5)	HEAD_Y
ParamEyeBallX	(t+offset, 6)	EYE_X
ParamEyeBallY	(t+offset, 7)	EYE_Y

3.1.4 天气表情联动

天气状态自动影响空闲表情基调:

表 2: 天气表情联动

天气	表情效果
晴/多云	ParamMouthForm 微笑 +0.2
雨/阴	ParamBrowRY/LY 眉抬 +4, MouthForm 微嘟 +0.2
雷暴	同上 + ParamEyeLOpen 微睁
雪	ParamMouthOpenY 微张 +0.4, EyeLOpen 微睁 +0.2
夜晚 (18-22 点)	EyeLOpen 垂 +0.07
深夜 (22-5 点)	EyeLOpen 垂 +0.15

3.2 DWM 透明窗口

窗口透明使用 Windows DWM API 实现:

```

1 // 1. 设置窗口样式
2 SetWindowLong(hwnd, GWL_EXSTYLE, WS_EX_LAYERED | WS_EX_TRANSPARENT);
3
4 // 2. 扩展玻璃框架 (黑色区域 = 透明)
5 MARGINS margins = new MARGINS { cxLeftWidth = -1 };
6 DwmExtendFrameIntoClientArea(hwnd, ref margins);
7
8 // 3. 设置背景纯黑透明
9 Camera.backgroundColor = new Color(0, 0, 0, 0);

```

Listing 2: DWM 玻璃层扩展

关键特性：

- 无绿边：DWM 玻璃层与纯黑背景配合，消除色键抠图的半透明边缘伪影
- 分层渲染：Live2D 渲染到 RenderTexture，通过 RawImage 显示在 Layer 31
- 点击穿透：每帧动态切换 WS_EX_TRANSPARENT
- DWM 崩溃保护：连续 5 次重建失败则跳过 DwmExtendFrameIntoClientArea

4 AI 对话系统

4.1 DeepSeek Chat API 集成

4.1.1 系统提示词注入

`ChatManager.BuildSystemPrompt()` 在每次对话前动态构建 System Prompt，注入内容包括：当前时间、法眼活动追踪摘要、浏览器标签页列表、长期记忆格式化文本、Live2D 参数语义知识、具身动作生成能力描述。

4.1.2 Function Calling 架构

37+ 个注册工具通过 JSON Schema 暴露给 DeepSeek，覆盖网页搜索、截图分析、系统控制、文件搜索（Everything 毫秒级）、便签管理、课表查询、动作/表情播放、长期记忆操作、应用/文件夹启动、用户状态查询等。

4.1.3 多轮工具调用循环

最多 5 轮的工具调用回环：

```
1 SendMessage()  
2     SendRequestCoroutine()  
3         1. 发送请求 (含 system prompt + 历史)  
4         2. 解析 response  
5         3. 如果有 tool_calls:  
6             检查 round < 5  
7             遍历 tool_calls:  
8                 IsCoroutineTool()? -> 协程异步执行  
9                 同步工具 -> Execute()  
10                收集结果, 回到 1 (下一轮)  
11         4. 无 tool_calls -> 处理回复文本
```

Listing 3: 多轮工具调用循环

4.1.4 异步工具执行

5 个可能耗时的工具通过协程异步执行，每帧检查 `task.IsCompleted`，不阻塞主线程：

- `query_exams / query_scores / query_schedule / query_user_status` — HTTP GET 请求课表小程序服务端
- `search_files` — Everything CLI 全盘搜索（毫秒级）或递归回退

4.2 工具索引

表 3: 工具列表 (37+)

工具	功能	类型
<code>get_time</code>	获取当前时间	同步
<code>open_url</code>	打开网页/文件/文件夹	同步
<code>open_app</code>	通过 PATH/StartMenu 启动应用	同步
<code>search_web</code>	联网搜索 (Bing)	同步
<code>get_weather</code>	查询天气	同步
<code>take_screenshot</code>	截屏 + GLM-4V 分析	协程异步
<code>set_volume / mute</code>	音量控制	同步
<code>get_system_info</code>	获取系统信息	同步
<code>lock_screen</code>	锁屏	同步
<code>system_power</code>	关机/重启/睡眠	同步 (需确认)
<code>run_command</code>	执行 CMD	同步 (需确认)
<code>get_clipboard / set_clipboard</code>	剪贴板读写	同步
<code>get_memories / write_memory</code>	长期记忆操作	同步
<code>start_conversation</code>	发起聊天	同步
<code>messenger</code>	模拟回复	同步
<code>write_note</code>	写便签	同步
<code>send_notification</code>	桌面通知	同步
<code>notify</code>	推送通知 (Server 酱 ³)	同步
<code>get_pet_status</code>	获取宠物状态	同步
<code>get_mouse_pos</code>	获取鼠标坐标	同步
<code>list_files</code>	列出目录文件	同步
<code>set_expression</code>	切换面部表情	同步
<code>play_action / stop_action</code>	播放/停止动作	同步
<code>open_folder</code>	打开资源管理器	同步
<code>query_exams</code>	查询考试安排	协程异步

表 3 续前页

工具	功能	类型
query_scores	查询成绩	协程异步
query_schedule	查询课表	协程异步
query_user_status	查询学业概览	协程异步
search_files	全盘文件搜索（Everything）	协程异步
inspect_motion_memory	查看动作记忆	同步
generate_motion	LLM 生成自定义动作	协程异步
get_system_status	获取系统状态	同步
show_reminder / delete_reminder	便签管理	同步

4.3 句子队列与优先级气泡

长回复被拆分为单个句子，以打字机风格逐句显示。ChatBubble 实现了三级优先级抢占机制：

表 4: 气泡优先级系统

优先级	用途	覆盖规则
Low	闲话、定时问候、待机气泡	可被任何高优消息覆盖
Normal	提醒、交互回应	不可被 Low 覆盖
High	AI 主动回复	不可被任何消息覆盖

4.4 自动闲聊系统

IdleChatGenerator 在无交互 60–120s 后触发，通过 DeepSeek API 批量预生成（每次 10 条），本地 JSON 缓存。按时间周期自动刷新缓存，API 不可用时回退硬编码问候语。

5 具身动作系统

5.1 架构演进

动作系统经过 10 个阶段的持续演化：

- **Phase 1–5:** 硬编码 10 个空闲动作（直接写在 Live2DRenderer 中）
- **Phase 6:** 动作参数提取为 KNOWN_PATTERNS 常量
- **Phase 7:** 7 个动作迁移到 JSON 配置 + IdleActionScheduler

- **Phase 8: MotionTranslator** — LLM 翻译自然语言到关键帧
- **Phase 9 (N32d): 闭环自评** — GLM-4V 视觉验证 + MotionMemoryManager 优化
- **Phase 10 (V2.1): 动作系统重构**到 Live2DFramework/ActionAgent/ 子目录, MotionAgent 引入自主动作决策 (4 级密度), IdleActionScheduler 增加时段/天气权重调整, 支持 12 种空闲动作

5.2 空闲动作系统

5.2.1 JSON 配置驱动

空闲动作定义在 `idle_actions.json` 中, 每个动作包含多阶段参数目标和插值曲线:

```
{
  "formatVersion": "v2",
  "actions": [{
    "id": 1,
    "name": "head_tilt",
    "displayName": "歪头",
    "weight": 10,
    "cooldown": 8.0,
    "phases": [
      { "duration": 0.5, "curve": "smooth",
        "targets": { "ParamAngleZ": 5 } },
      { "duration": 0.5, "curve": "smooth",
        "targets": { "ParamAngleZ": 0 } }
    ]
  }]
}
```

Listing 4: 空闲动作 JSON 配置

5.2.2 动作列表

表 5: 空闲动作列表

ID	名称	权重	类型
1	歪头	10	JSON
2	微笑	10	JSON
3	挑眉	8	JSON
4	星辉缠绕	5	硬编码
5	伸懒腰	8	JSON
6	委屈	6	JSON
7	法阵展开	4	硬编码
8	害羞	6	JSON
9	困惑	0	JSON (AI 仅用)
10	爱心	5	JSON
11	哭	4	JSON
12	心跳	5	JSON

5.2.3 插值曲线

MotionGenerator 支持 6 种插值曲线:

linear: $f(t) = t$
smooth: $f(t) = 3t^2 - 2t^3$
easeOut: $f(t) = 1 - (1 - t)^2$
easeIn: $f(t) = t^2$
hold: $f(t) = 0$
bounce: 分段弹性曲线

5.3 LLM 动作翻译 (MotionTranslator)

5.3.1 核心流程

1. 构建 Body Schema — 所有可用参数名、范围、语义描述
2. 注入运动记忆 — MotionMemoryManager 提供历史成功模板
3. 调用 DeepSeek API (temperature=0.3, 超时 30s)
4. 解析 JSON 响应 → MotionPlan (关键帧序列)
5. MotionGenerator 协程逐帧插值播放

5.3.2 SPECIAL PATTERNS

内嵌在 System Prompt 中的姿势模板，帮助 DeepSeek 准确生成常见姿势：

表 6: SPECIAL PATTERNS

模式	关键参数
叉腰	arm_right_upper=0.7 0.8, arm_right_lower=0.15 0.25
行礼	body_angle_x=15 25, arm_right_lower=0.5 0.7
歪头思考	head_angle_z=15 25, eye_ball_y=-0.5 -0.7
捂脸	arm_right_upper=0.8 1.0, hand_near_face=1.0
惊讶	head_angle_y=8 15, eye_l_open=0.9 1.0
缩团	head_angle_y=-15 -25, arm 夹紧

5.4 动作锁定机制

AI 生成动作播放期间，通过 `_aiControlLocked` 和 `_actionLocked` 双锁防止被空闲动作或走路覆盖：前者阻止 Perlin 噪声和天气表情更新，后者阻止空闲动作调度器和拖拽动画。

5.5 MotionAgent — 自主动作决策

`MotionAgent` 在 V2.1 中引入，负责自主判断何时执行动作、执行什么动作，基于本地 LLM（Qwen2.5:0.5b）决策：

- 4 级动作密度：High / Medium / Low / Sleep，根据当前焦点进程动态调整
- 焦点进程抑制：检测到全屏游戏或编辑器在前台时自动降低动作频率
- 情绪状态机：EmotionState 维护基础情绪（neutral/happy/sad/angry/surprised），影响动作选择
- 空闲弧检测：长时间无交互时触发更频繁的随机动作
- 本地决策：通过 LocalLLMClient 调用 Ollama，延迟可控

6 闭环具身智能验证

6.1 架构设计

闭环具身智能验证系统通过”生成 → 执行 → 观察 → 评估 → 优化”的循环，使 AI 的动作能力持续改进：

DeepSeek(生成) → Live2D(执行) → 截图 (捕获) → GLM-4V(评估) → 记忆 (学习)

6.2 MotionVerifier — 动作验证器

三级测试集共 19 个测试用例：

- **Level 1 (对照组)**：5 个应走硬编码模板的动作（挥手、点头、摇头、鞠躬、伸懒腰）
- **Level 2 (测试组)**：10 个应走 LLM 翻译路径的动作（害羞捂脸、叉腰、惊讶捂嘴、忧郁远望、俏皮眨眼、行礼、缩团、骄傲抬头、歪头思考、合十祈祷）
- **Level 3 (边界组)**：4 个边界输入（空字符串、乱码、复杂指令、物理不可能指令）

评估指标包括：参数合规率、对称配比率、关键帧数、复位帧检测。

6.3 VisionMotionVerifier — 视觉验证器

对每个测试用例执行完整流程：

1. 走 LLM 路径生成动作
2. 播放动作至峰值帧
3. 截取 RenderTexture 为 PNG
4. 发送截图 + 评价提示给 GLM-4V
5. 解析 GLM 返回的评分 (1-5) 和评语

GLM-4V 评价标准：

$$\text{Score} = \begin{cases} 5, & \text{非常像, 动作精准传达意图} \\ 4, & \text{比较像, 主要特征符合} \\ 3, & \text{一般, 部分特征符合} \\ 2, & \text{不太像, 很多特征缺失} \\ 1, & \text{完全不像} \end{cases}$$

6.4 DualModelValidator — 双模型评分

V2.1 中采用 GLM-4V 作为主要评分模型 (pass 阈值 3/5), Qwen-VL-Plus 作为只记录不影响的辅助评分:

- **GLM-4V:** 主评分器, 返回 1-5 分 + 文本评语
- **Qwen-VL-Plus:** 仅用于日志记录, 不参与结果判断

6.5 闭环学习 (MotionMemoryManager)

视觉验证结果反馈到 MotionMemoryManager:

- 评分 ≥ 3 (通过阈值): 加入运动记忆库 (上限 30 条), 供后续生成参考
- 评分 < 3 : 记录到 VERIFIED FEEDBACK 列表 (上限 10 条), 作为负面示例附带
- **无望检测:** 同一动作 5 次尝试评分均 $\leq 2/5$ 时标记为「无望」, 不再尝试
- 记忆注入优先级: 高评分 $>$ 低评分, 新注入按分数排序
- 下次 DeepSeek 调用时附带上一次 FEEDBACK 作为负面示例, 引导改进

7 交互系统

7.1 拖拽系统 (DragHandler)

DragHandler 管理所有交互与点击穿透的协调。每帧根据鼠标位置和面板状态动态设置 WS_EX_TRANSPARENT 或清除该标记:

- 鼠标在 BallPanel 交互区域 $\rightarrow$ 关闭穿透 (允许点击面板)
- 鼠标在 RightPanel 交互区域 $\rightarrow$ 关闭穿透 (允许点击按钮/输入)
- 面板关闭时 $\rightarrow$ 强制重置穿透状态
- 拖拽释放时计算速度向量传递给 DesktopPet 物理引擎

拖拽释放时的抛掷物理:

$$\mathbf{v}_{\text{throw}} = \frac{\sum_{i=1}^n (\mathbf{p}_i - \mathbf{p}_{i-1})}{n \cdot \Delta t}, \quad |\mathbf{v}_{\text{throw}}| \leq \text{maxThrowSpeed} = 12$$

抛掷速度经多帧缓冲平均后限幅, 防止瞬间抖动的异常高速。

7.2 悬浮球辐射菜单 (BallPanel)

V2.1 全新引入的交互方式。桌面右下角粉色 悬浮球，单击展开辐射菜单：

- 设置面板：任务权重滑块调节 + 保存/清空，420×580px 浮动窗口
- 报告面板：MotionMemoryManager 演武心经学习报告展示
- 便签面板：待办/已完成管理，增删改查操作

面板通过 OnGUI 渲染，使用自定义 GUIStyle。标题栏可拖拽移动，右键关闭面板。PanelType 枚举管理三种面板状态（None/Settings/Report/Reminders）。

7.3 右侧 Widgets 面板 (RightPanel)

Windows 11 Widgets 风格右侧滑出面板：

- ~ 键切换展开/收起，鼠标划过右边缘自动展开
- 1 秒自动隐藏延迟（鼠标离开后计时）
- 展开宽度 220px，收起宽度 8px，滑动速度 10f/s
- 4 个工具按钮：聊（聊天聚焦）/ 设（打开设置）/ 签（打开便签）/ 告（打开报告）
- 底部集成输入栏

IsPointInInteractiveArea(Vector2) 供 DragHandler 判断点击穿透状态。

7.4 分区点击反馈

通过 CubismRaycaster 实现分区检测：

- Head (高度 > 0.6)：摸头眯眼锁定动画
- Body (0.3 0.6)：捂胸惊讶
- Leg (< 0.3)：害羞踢腿

7.5 拖拽挣扎动画

拖拽过程中触发挣扎动画：双臂交替划水 + 双腿交替 + 身体扭动 + 慌张表情。帧间速度经平滑滤波后直接写入 Cubism 参数，绕过物理系统的 0.8s 延迟，实现即时响应。

拖拽时 20+ 头发输出参数由帧间速度直接驱动，裙子（Param82/87/84）、法盘（Param49/51/57/60）同步跟随，实现物理惯性拖尾效果。

8 感知与记忆系统

8.1 法眼活动追踪 (ActivityTracker)

每 2 秒通过 `GetForegroundWindow()` + `GetWindowText()` 获取前台窗口标题，按关键词分类为编程、设计、游戏、学习、浏览、办公、通讯、音乐等。

通过 `EnumWindows` 扫描所有可见顶层窗口，构建多窗口环境摘要。`BrowserTabReader` 通过 UI Automation API 读取主流浏览器标签页标题 (Chrome/Edge/Firefox/Opera/Brave/Vivaldi)，无需安装插件。

8.2 长期记忆系统 (PetMemory)

8.2.1 三层分层结构

- **核心事实**：用户偏好、重要承诺、关键信息（始终保留）
- **重要记忆 (Top-5)**：按重要性分数排序
- **近期琐事**：时间衰减，自动淘汰

记忆总量上限 30 条。核心事实永久保留，超出上限时优先淘汰低分琐事。

8.2.2 记忆类别与重要性

表 7: 记忆类别与初始重要度

类别	说明	初始重要度
tool	工具调用记录	0.3
conversation	对话摘要	0.5
observation	法眼观察	0.2
reflection	反思提炼	0.8

8.2.3 反思反射

累计重要性积分达阈值（默认 30）时触发 LLM 提炼高层次洞察：

$$\sum \text{importance} \geq \text{THRESHOLD} = 30 \Rightarrow \text{TriggerReflection}()$$

同话题在冷却期间（默认 120s）不再重复记录。反思结果按 `reflection` 类别存入核心层，初始重要度 0.8。

8.3 时间与天气系统 (TimeWeatherController)

8.3.1 双源天气获取

QWeather API 优先 (需 Key)，失败时自动回退 wttr.in (无 Key 要求)。解析天气类型: Clear / Cloudy / Overcast / Rain / Drizzle / Thunder / Snow / Fog。

8.3.2 AI 天气语录

当 QWeather 可用时，调用 DeepSeek 批量生成符玄风格的天气台词，按天气类型缓存，随天气变化自动刷新。

9 桌面物理引擎

9.1 核心物理模型

简化的 2D 像素物理引擎：

$$v_y = v_y + g \cdot \Delta t$$

$$y = y + v_y \cdot \Delta t$$

$$\mathbf{v} = \mathbf{v} \cdot \text{AIR_RESISTANCE}$$

地面碰撞：触地后速度反向乘以反弹系数，低于阈值时停止。

9.2 地面任务状态机

5 种地面行为通过加权随机切换：

表 8: 地面任务状态机

任务	权重 (默认)	行为
MoveLeftEdge	1	向左走到屏幕左边缘
MoveRightEdge	1	向右走到屏幕右边缘
MoveLeftTime	1	向左走随机时长 (2–4 s)
MoveRightTime	1	向右走随机时长 (2–4 s)
StopTime	6	停止随机时长 (6–15 s)

9.3 多显示器支持

通过 VirtualScreen 获取跨越所有显示器的完整桌面区域，物理边界自动适配多显示器环境。

9.4 睡眠唤醒检测

通过帧间时间间隙检测系统休眠/唤醒：

$$\Delta t_{\text{gap}} > \text{SLEEP_DETECT_THRESHOLD} \Rightarrow \text{RebuildDWM}() + \text{ResetPhysics}()$$

10 窗口与系统集成

10.1 Win32 API 封装

10.1.1 DWM 透明窗口

窗口透明使用 `DwmExtendFrameIntoClientArea`，MARGINS 结构 `cxLeftWidth = -1` 表示全窗口玻璃层扩展。Camera 背景统一为纯黑 (0,0,0,0)。

10.1.2 系统托盘

使用 `Shell_NotifyIcon` 实现系统托盘图标，支持左键切换显示/隐藏、右键弹出菜单（显示/隐藏/开机自启/退出）。

10.1.3 开机自启

通过注册表实现：

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

10.2 崩溃日志系统

通过 `Application.logMessageReceived` 捕获异常和错误，追加到本地日志文件。日志长度超限时自动截断（保留后半部分），防日志爆炸。

10.3 进程互斥

使用 `Mutex`（`DesktopPet_Unity_SingleInstance`）防止多实例。

11 性能优化

11.1 三级性能档位

`PerformanceMonitor` 管理三级 FPS 自适应档位：

表 9: 性能档位

档位	目标帧率	RT 缩放	触发条件
High	60 fps	100%	FPS > 45 且 CPU/GPU < 80%
Normal	40 fps	75%	FPS 30–45 或 CPU/GPU < 85%
Low	20 fps	50%	FPS < 30 或 CPU/GPU > 92%

11.2 帧率采样与评估

滚动 90 帧采样窗口，每 30 帧评估一次性能，决定是否升降档。

11.3 CPU/GPU 监控

- **CPU**: 通过 Win32 `GetSystemTimes` 获取总占用率
- **GPU**: 通过 NVML (NVIDIA Management Library) 获取 GPU 占用率
- **升档拦截**: CPU/GPU 占用 > 80% 时阻止升档
- **紧急降级**: CPU/GPU 占用 > 92% 时强制降至 Low 档

11.4 内存管理

$$\left\{ \begin{array}{ll} \text{appMemory} > 800 \text{ MB}, & \text{GC.Collect}() \\ \text{appMemory} > 1200 \text{ MB}, & \text{GC.Collect}(2, \text{Forced}) \\ \text{systemMemory} > 85\%, & \text{Resources.Unload} + \text{GC} \\ \text{systemMemory} > 93\%, & \text{强制 Low 档} + \text{GC} \end{array} \right.$$

系统总内存通过 `GlobalMemoryStatusEx` 获取,应用内存通过 `Profiler.GetTotalAllocatedMemory` 获取。

12 便签与提醒系统

12.1 数据模型

支持一次性、每日、工作日、每周四种重复规则。所有提醒序列化为 JSON 持久化到本地。

12.2 推送链路

到期提醒通过三级推送链路依次触发：头顶气泡显示 → Windows Toast 通知 → Server 酱³ 手机推送。

12.3 去重机制

AI 手动设置与服务器推送的同类提醒自动去重。创建前通过 `HasPendingReminderContaining()` 检查关键词，防止重复。

13 构建与部署

13.1 构建脚本

标准构建使用 `build.ps1`，支持 `-Quick` 参数仅验证编译（不输出可执行文件）：

```
1 public static void BuildDesktopPet() {
2     string[] scenes = EditorBuildSettings.scenes
3         .Where(s => s.enabled).Select(s => s.path).ToArray();
4
5     BuildPlayerOptions options = new BuildPlayerOptions {
6         scenes = scenes,
7         locationPathName = "Build/DesktopPet.exe",
8         target = BuildTarget.StandaloneWindows64,
9         options = BuildOptions.None
10    };
11
12    BuildPipeline.BuildPlayer(options);
13 }
```

Listing 5: BuildScript.cs 核心逻辑

13.2 构建后验证

- 关键文件检查：Build/DesktopPet.exe、Assembly-CSharp.dll
- 构建日志关键字搜索：error、warning、构建完成

14 版本路线图

表 10: 版本演进

版本	日期	主要变更
v0.2	—	PNG 渲染初步 + API Key 环境变量
v0.3	—	10 种空闲动作 + 加权随机选择
v0.4	—	右键菜单 + 权重编辑
v0.5	—	动作锁定 + 走路淡入过渡
v0.6	2026-06-13	LaTeX 文档 + 右键菜单完善
v0.7	2026-06-17	修复物理覆盖 + 执行顺序修正
v0.8	2026-06-17	物理直接驱动 + 分区点击 + 挣扎动画 + 天气响应
v0.9	2026-06-18	DWM 玻璃层透明 + 底部输入栏
N10	2026-06-18	点击穿透 + 右键菜单重构 + 多轮工具调用
N11	2026-06-18	优先级气泡系统 + AI 回复稳定
N12	2026-06-19	性能监控 + 开机自启
N13	2026-06-20	服务端推送轮询 + Server 酱 ³ 集成
N14	2026-06-20	小程序课表数据打通 + 3 个学业查询工具
N15	2026-06-20	修复逐句显示 bug
N16	2026-06-20	Everything 毫秒级文件搜索
N17	2026-06-22	提醒去重 + 服务器推送去重
N18	2026-06-22	已完成任务独立视图 + 系统总内存监控
N32d	2026-07-07	闭环具身智能 + GLM-4V 视觉验证 + 运动记忆
V2.1	2026-07-09	悬浮球 + 辐射菜单—Ball-Panel+RightPanel 取代右键菜单；动作系统重构到 Live2DFramework/ActionAgent/

15 附录

15.1 环境变量清单

表 11: 环境变量

变量名	必需	用途
DEEPSEEK_API_KEY	是	DeepSeek Chat API
GLM_API_KEY	否	GLM-4V 截图分析与动作验证
QWEATHER_API_KEY	否	和风天气数据源

15.2 相关文件路径

表 12: 文件路径

用途	路径
构建日志	code/desktop_unity/build_log6.txt
崩溃日志	Build/crash_log.txt
长期记忆	{persistentDataPath}/pet_memory.json
天机簿配置	{persistentDataPath}/pet_config.json
提醒数据	{persistentDataPath}/reminders.json
系统提示词	Assets/Resources/SystemPrompt.txt
空闲动作配置	Assets/Resources/IdleActions/idle_actions.json
浮游法阵图	Assets/Resources/magic_circle.png

15.3 关键设计决策

表 13: 设计决策

决策	理由
DWM 玻璃层而非色键	彻底解决半透明绿边问题
DeepSeek Chat（非本地）	高质量对话 + Function Calling 原生支持
GLM-4V（非 GPT-4V）	国内可用 + 图像理解质量达标
LLM 翻译关键帧	无限动作多样性 vs 硬编码有限集
JSON 配置驱动空闲动作	扩展性，无需改代码即可添加新动作
三层分层记忆	核心稳定 + 重要不丢 + 琐事自动淘汰
QWeather 优先 + wttr.in 回退	准确性 + 无需注册的双保险
Everything CLI 优先	毫秒级 vs 分钟级的巨大差距
悬浮球 + 右侧面板取代右键菜单	更好利用屏幕空间，右下方悬浮不遮挡桌面内容
4 级密度动作决策	根据焦点进程抑制空闲动作，减少 AI 频繁介入干扰